2.4 Keeping state with behaviors

Keeping state with different behaviors in an actor has a different purpose than storing state with variables. You use behaviors to represent that the actor reacts differently to the same messages. If you only want to change variables within the actor, don't use this method; use what you learned in the previous section.

Depending on the actor's state, the same message can do very different things. For example, an actor can act as a proxy and have two states: open and closed. When it is open, it forwards messages to another destination, as any proxy would. However, when it is closed, it does not forward messages: instead, it notifies a third party.

Let's implement another wallet. Similar to the previous variant, for the sake of simplicity, you can increase the amount but not decrease it. What is new is that this wallet has two states represented by two behaviors: activated and deactivated. If the wallet is activated, it responds as in the previous example and increases the total amount when it receives an `Increase`. On the other hand, if the wallet is deactivated, the total amount cannot be changed. If the actor receives an `Increase` in this state, it discards the `Increase` and reports that no change was made.

NOTE This feature makes this actor a finite state machine (FSM) because it reacts differently to the same input. Without going into too much detail, an FSM is a model that has a finite number of states. Depending on the state—also called a behavior—the same input can produce different results. Because actors are defined by behaviors, they are perfect for implementing this model.